INDUSTRIAL ASSIGNMENT- 11 Developing Backend code using Spring Boot
STUDENT INTERNSHIP AND PLACEMENT MANAGEMENT PORTAL

1. Introduction

Industrial Assignment 10 described the foundation of the Spring Boot backend: architecture, configuration, entities, repositories, DTOs and security. This assignment explains the remaining layers that make the backend work: the services that apply the business rules, the controllers that expose the REST APIs, the validation and exception handling, the dashboard and reports, and the way the Next.js frontend and the automated test use the APIs. The assignment ends with a table that shows how the 28 tasks of the project are covered.

2. Objectives

- Implement the business rules of the portal in service classes with transactions.
- Expose REST APIs under /api with correct HTTP methods and status codes.
- Restrict every API by role and make sure students and recruiters see only their own data.
- Return clear JSON errors for validation failures, missing records and conflicts.
- Provide a live dashboard and report summary for the Placement Officer and staff.
- Verify the backend with an automated integration test before Postman testing.

3. Service Layer

Each service class is annotated with @Service and @Transactional. Read methods use @Transactional(readOnly = true). A service receives the DTO from the controller, loads or saves entities through repositories, checks the rules and returns a response DTO.

Service	Responsibility
StudentService	List, view, create, update and delete students; find the profile of the logged-in student by e-mail; reject duplicate student ID or e-mail.
CompanyService	List, view, create, update and delete companies; reject duplicate company e-mail; e-mail is stored in lower case.
OpportunityService	List (all or by type), view, create, update and delete opportunities; links each opportunity to an existing company.
ApplicationService	Apply for an opportunity, list applications by role, update the status and delete an application; enforces the application rules in Section 4.
InterviewService	Schedule, list, update and delete interviews; a recruiter works only with interviews of the own company; counts upcoming interviews.

Service	Responsibility
DashboardService	Calculates totals, stage counts and recent activity from live data for the dashboard and report APIs.

3.1 Student service: create and update

The service trims the input, converts the e-mail to lower case and checks for duplicates before saving. For an update, the "IdNot" repository methods allow the record to keep its own values.

```
    public StudentResponse create(StudentRequest request) {
        String studentCode = request.studentCode().trim();
        String email = request.email().trim().toLowerCase(Locale.ROOT);
        if (repository.existsByStudentCode(studentCode) || repository.existsByEmail(email))
{
            throw new ConflictException("Student ID or email is already registered.");
        }
        Student student = new Student();
        apply(student, request);
        return toResponse(repository.save(student));
    }

    public StudentResponse update(Long id, StudentRequest request) {
        Student student = getEntity(id);
        String studentCode = request.studentCode().trim();
        String email = request.email().trim().toLowerCase(Locale.ROOT);
        if (repository.existsByStudentCodeAndIdNot(studentCode, id)
                || repository.existsByEmailAndIdNot(email, id)) {
            throw new ConflictException("Student ID or email is already registered.");
        }
        apply(student, request);
        return toResponse(student);
    }
```

3.2 Application service: apply

Applying is the most important rule of the portal. The service finds the student and the opportunity, checks that a student applies only for himself or herself, that the opportunity is still open and that the student has not applied before.

```
    public ApplicationResponse create(ApplicationRequest request, String username, String
role) {
        Student student = students.findById(request.studentId())
                .orElseThrow(() -> new ResourceNotFoundException("Student",
request.studentId()));
        if ("ROLE_STUDENT".equals(role) &&
!student.getEmail().equalsIgnoreCase(username)) {
```

```
            throw new AccessDeniedException("Students may only apply using their own
account.");
        }
        Opportunity opportunity = opportunities.getEntity(request.opportunityId());
        if (opportunity.getStatus() != OpportunityStatus.ACTIVE ||
opportunity.getDeadline().isBefore(LocalDate.now())) {
            throw new ConflictException("Applications are closed for this opportunity.");
        }
        if (applications.existsByStudentIdAndOpportunityId(student.getId(),
opportunity.getId())) {
            throw new ConflictException("This student has already applied to this
opportunity.");
        }
        StudentApplication application = new StudentApplication();
        application.setStudent(student);
        application.setOpportunity(opportunity);
        application.setStatus(ApplicationStatus.APPLIED);
        return toResponse(applications.save(application));
    }
```

3.3 Application service: list and update status

The list method returns different data for each role. A student gets the own applications, a recruiter gets the applications of the company whose e-mail equals the login name, and staff get all applications. The status can be changed only by staff or by the recruiter of the owning company.

```
    public List<ApplicationResponse> findAll(String username, String role) {
        List<StudentApplication> results = switch (role) {
            case "ROLE_STUDENT" ->
applications.findAllByStudent_EmailOrderByAppliedAtDesc(username);
            case "ROLE_RECRUITER" ->
applications.findAllByOpportunity_Company_EmailOrderByAppliedAtDesc(username);
            default -> applications.findAllByOrderByAppliedAtDesc();
        };
        return results.stream().map(ApplicationService::toResponse).toList();
    }
```

```
    public ApplicationResponse updateStatus(Long id, ApplicationStatusRequest request,
String username, String role) {
        StudentApplication application = getEntity(id);
        if ("ROLE_STUDENT".equals(role)) {
            throw new AccessDeniedException("Students cannot change an application
status.");
        }
```

```
        if ("ROLE_RECRUITER".equals(role)
                &&
!application.getOpportunity().getCompany().getEmail().equalsIgnoreCase(username)) {
            throw new AccessDeniedException("Recruiters may only update applications for
their company.");
        }
        application.setStatus(request.status());
        return toResponse(application);
    }
```

4. Business Rules Implemented

Rule	Implementation in the code	Result when broken
R1	A student can apply only with the own account (student e-mail must equal the login name).	403 Forbidden
R2	An application is accepted only if the opportunity status is ACTIVE and the deadline has not passed.	409 Conflict: "Applications are closed for this opportunity."
R3	A student can apply to an opportunity only once (service check and unique database constraint).	409 Conflict: "This student has already applied to this opportunity."
R4	A new application always starts with status APPLIED.	-
R5	Student ID and student e-mail are unique; company e-mail is unique.	409 Conflict
R6	Students see only ACTIVE opportunities whose deadline is not over; closed or expired ones return 404.	404 Not Found
R7	A recruiter may create, edit or delete opportunities only for the company whose contact e-mail equals the login name.	403 Forbidden
R8	A recruiter may update application status and manage interviews only for the own company.	403 Forbidden
R9	Only staff may change student and company records, view the dashboard and reports and delete applications.	403 Forbidden

Rule	Implementation in the code	Result when broken
R10	A record that is still referenced (company with opportunities, student with applications, opportunity with applications) cannot be deleted.	409 Conflict

5. Controllers and REST API

Controllers are thin: they map the URL, read the logged-in user from the Authentication object, validate the body with @Valid and call the service. Successful creation returns 201 Created with a Location header; deletion returns 204 No Content. All endpoints begin with /api and exchange JSON.

Method	Endpoint	Allowed roles	Success response
GET	/api/auth/me	Any logged-in user	200 user name and roles
GET	/api/students	STAFF	200 list
GET	/api/students/me	STUDENT	200 own profile
GET	/api/students/{id}	STAFF	200 student
POST	/api/students	STAFF	201 created
PUT	/api/students/{id}	STAFF	200 updated
DELETE	/api/students/{id}	STAFF	204 no content
GET	/api/companies, /api/companies/{id}	STUDENT, RECRUITER, STAFF	200
POST, PUT, DELETE	/api/companies, /api/companies/{id}	STAFF	201, 200, 204
GET	/api/opportunities?type=	STUDENT, RECRUITER, STAFF	200 list (students: active only)
GET	/api/opportunities/{id}	STUDENT, RECRUITER, STAFF	200 opportunity
POST, PUT, DELETE	/api/opportunities, /{id}	STAFF, RECRUITER (own company)	201, 200, 204
GET	/api/applications	STUDENT, RECRUITER, STAFF	200 list in the scope of the role
POST	/api/applications	STUDENT (own), STAFF	201 created
PUT	/api/applications/{id}/status	STAFF, RECRUITER (own company)	200 updated

Method	Endpoint	Allowed roles	Success response
DELETE	/api/applications/{id}	STAFF	204 no content
GET, POST	/api/interviews	STAFF, RECRUITER (own company)	200, 201
PUT, DELETE	/api/interviews/{id}	STAFF, RECRUITER (own company)	200, 204
GET	/api/dashboard	STAFF	200 live overview
GET	/api/reports	STAFF	200 live summary

5.1 Example controller: ApplicationController

```
@RestController
@RequestMapping("/api/applications")
public class ApplicationController {
   private final ApplicationService service;

   public ApplicationController(ApplicationService service) {
      this.service = service;
   }

   @GetMapping
   public List<ApplicationResponse> findAll(Authentication authentication) {
      return service.findAll(authentication.getName(), role(authentication));
   }

   @PostMapping
   @PreAuthorize("hasAnyRole('STUDENT', 'STAFF')")
   public ResponseEntity<ApplicationResponse> create(@Valid @RequestBody
ApplicationRequest request,
         Authentication authentication) {
      ApplicationResponse result = service.create(request, authentication.getName(),
role(authentication));
      return ResponseEntity.created(URI.create("/api/applications/" +
result.id())).body(result);
   }

   @PutMapping("/{id}/status")
   @PreAuthorize("hasAnyRole('STAFF', 'RECRUITER')")
   public ApplicationResponse updateStatus(@PathVariable Long id,
         @Valid @RequestBody ApplicationStatusRequest request, Authentication
authentication) {
      return service.updateStatus(id, request, authentication.getName(),
role(authentication));
   }
```

```
    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('STAFF')")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
    // ... role(authentication) helper
}
```

5.2 Ownership check in OpportunityController

Before a recruiter creates, edits or deletes an opportunity, the controller compares the contact e-mail of the company with the login name. A mismatch throws AccessDeniedException, which Spring Security turns into HTTP 403.

```
    private void verifyRecruiterCompany(Long companyId, Authentication authentication) {
        boolean recruiter =
authentication.getAuthorities().stream().map(GrantedAuthority::getAuthority)
                .anyMatch("ROLE_RECRUITER"::equals);
        if (recruiter &&
!companies.getEntity(companyId).getEmail().equalsIgnoreCase(authentication.getName()))
{
            throw new AccessDeniedException("Recruiters may only manage opportunities for
their company.");
        }
    }
```

5.3 Filtering by opportunity type

```
    public List<OpportunityResponse> findAll(@RequestParam(required = false)
OpportunityType type,
            Authentication authentication) {
        boolean student =
authentication.getAuthorities().stream().map(GrantedAuthority::getAuthority)
                .anyMatch("ROLE_STUDENT"::equals);
        return service.findAll(type).stream().filter(opportunity -> !student
                || (opportunity.status() == OpportunityStatus.ACTIVE &&
!opportunity.deadline().isBefore(LocalDate.now())))
                .toList();
    }
```

6. Role Access Matrix

Area	STAFF	STUDENT	RECRUITER	Not logged in

Area	STAFF	STUDENT	RECRUITER	Not logged in
Own login check (/api/auth/me)	Yes	Yes	Yes	401
Students: list, view, create, update, delete	Yes	No (403)	No (403)	401
Own profile (/api/students/me)	No (403)	Yes	No (403)	401
Companies: view	Yes	Yes	Yes	401
Companies: create, update, delete	Yes	No (403)	No (403)	401
Opportunities: view	All	Active only	All	401
Opportunities: create, update, delete	Yes	No (403)	Own company	401
Applications: apply	Yes	Own account	No (403)	401
Applications: list	All	Own	Own company	401
Applications: update status	Yes	No (403)	Own company	401
Applications: delete	Yes	No (403)	No (403)	401
Interviews: all actions	Yes	No (403)	Own company	401
Dashboard and reports	Yes	No (403)	No (403)	401

7. Validation and Exception Handling

Input is validated twice: Bean Validation annotations on the DTOs check format and length before the controller method runs, and the services check rules that need the database. All errors are returned by ApiExceptionHandler, a @RestControllerAdvice class, in the same JSON shape: timestamp, status, error, message, path and fieldErrors.

Situation	HTTP code	Handled by
Missing or invalid field in the request body	400	MethodArgumentNotValidException handler, with fieldErrors per field
Unknown enum value, wrong type or broken JSON	400	Spring default handling
No login or wrong password	401	Spring Security (HTTP Basic)

Situation	HTTP code	Handled by
Logged in but role not allowed, or not the owner	403	Spring Security / AccessDeniedException
Record id does not exist	404	ResourceNotFoundException handler
Duplicate record, closed opportunity, duplicate application	409	ConflictException handler
Foreign-key or unique-key violation in MySQL	409	DataIntegrityViolationException handler

```
@RestControllerAdvice
public class ApiExceptionHandler {
   @ExceptionHandler(ResourceNotFoundException.class)
   ResponseEntity<Map<String, Object>> notFound(ResourceNotFoundException
exception, HttpServletRequest request) {
      return error(HttpStatus.NOT_FOUND, exception.getMessage(),
request.getRequestURI(), Map.of());
   }

   @ExceptionHandler(ConflictException.class)
   ResponseEntity<Map<String, Object>> conflict(ConflictException exception,
HttpServletRequest request) {
      return error(HttpStatus.CONFLICT, exception.getMessage(), request.getRequestURI(),
Map.of());
   }

   @ExceptionHandler(DataIntegrityViolationException.class)
   ResponseEntity<Map<String, Object>> constraint(DataIntegrityViolationException
exception, HttpServletRequest request) {
      return error(HttpStatus.CONFLICT, "The request conflicts with an existing record or
relationship.",
            request.getRequestURI(), Map.of());
   }

   @ExceptionHandler(ConstraintViolationException.class)
   ResponseEntity<Map<String, Object>>
constraintViolation(ConstraintViolationException exception,
         HttpServletRequest request) {
      return error(HttpStatus.BAD_REQUEST, "Request validation failed.",
request.getRequestURI(), Map.of());
   }

   @ExceptionHandler(MethodArgumentNotValidException.class)
```

```
    ResponseEntity<Map<String, Object>> invalid(MethodArgumentNotValidException
exception,
            HttpServletRequest request) {
        Map<String, String> fields = exception.getBindingResult().getFieldErrors().stream()
                .collect(Collectors.toMap(FieldError::getField, FieldError::getDefaultMessage,
(first, second) -> first));
        return error(HttpStatus.BAD_REQUEST, "Request validation failed.",
request.getRequestURI(), fields);
    }

    // ... error(status, message, path, fields) builds the JSON body
}
```

Example of an error response for a student created with missing fields:

```
{
  "timestamp": "2026-10-26T10:15:30",
  "status": 400,
  "error": "Bad Request",
  "message": "Request validation failed.",
  "path": "/api/students",
  "fieldErrors": {
    "name": "must not be blank",
    "email": "must not be blank"
  }
}
```

8. Dashboard and Reports

DashboardService calculates all figures from the database each time they are requested, so the dashboard and the reports always show the current data (NFR-06). The reports API returns the same live summary as the dashboard.

Field in DashboardResponse	Meaning
totalStudents, totalCompanies	Number of student and company records
activeInternships, activePlacements	Number of ACTIVE opportunities of each type
totalApplications, selectedStudents	All applications and applications with status SELECTED
upcomingInterviews	Interviews scheduled after the current time
applicationsByStatus	Count of applications for each of the six statuses
opportunitiesByType	Active opportunities by INTERNSHIP and PLACEMENT

Field in DashboardResponse	Meaning
placementOutcomes	Application statuses of PLACEMENT opportunities only
recentOpportunities, recentApplications	The five most recent records of each kind

```
   public DashboardResponse getOverview() {
      Map<String, Long> byStatus = new LinkedHashMap<>();
      for (ApplicationStatus status : ApplicationStatus.values()) {
         byStatus.put(status.name(), applications.countByStatus(status));
      }
      Map<String, Long> byType = new LinkedHashMap<>();
      for (OpportunityType type : OpportunityType.values()) {
         byType.put(type.name(), opportunities.countByTypeAndStatus(type,
OpportunityStatus.ACTIVE));
      }
      Map<String, Long> placementOutcomes = new LinkedHashMap<>();
      for (ApplicationStatus status : ApplicationStatus.values()) {
         placementOutcomes.put(status.name(),
               applications.countByOpportunityTypeAndStatus(OpportunityType.PLACEMENT,
status));
      }
      List<OpportunityResponse> recentOpportunities = opportunities

.findAllByOrderByCreatedAtDesc().stream().limit(5).map(OpportunityService::toResponse).toList();
      List<ApplicationResponse> recentApplications =
applications.findAllByOrderByAppliedAtDesc().stream()
            .limit(5).map(ApplicationService::toResponse).toList();
      return new DashboardResponse(students.count(), companies.count(),
            opportunities.countByTypeAndStatus(OpportunityType.INTERNSHIP,
OpportunityStatus.ACTIVE),
            opportunities.countByTypeAndStatus(OpportunityType.PLACEMENT,
OpportunityStatus.ACTIVE),
            applications.count(), applications.countByStatus(ApplicationStatus.SELECTED),
interviews.countUpcoming(),
            byStatus, byType, placementOutcomes, recentOpportunities, recentApplications);
   }
```

9. Frontend Integration

The frontend is a Next.js, React and TypeScript application (Tailwind CSS, Recharts for charts). The file src/lib/api.ts is the only place that talks to the backend. After login it calls GET /api/auth/me with the Basic authorization value; if the backend answers 200, the value is kept in the browser session storage and added as the Authorization header to every later request.

Signing out clears the stored value. API errors are shown in the interface by reading fieldErrors or message from the JSON error body.

Frontend page	Backend API used
/login	GET /api/auth/me
/ (dashboard)	GET /api/dashboard
/students	/api/students
/companies	/api/companies
/internships, /placements	/api/opportunities?type=INTERNSHIP and PLACEMENT
/applications	/api/applications and /api/applications/{id}/status
/interviews	/api/interviews
/reports	GET /api/reports

The browser may call the backend because CORS allows the frontend origin (http://localhost:3000 by default) for the methods GET, POST, PUT, DELETE and OPTIONS.

10. Automated Integration Test

The class SipmpApiIntegrationTest starts the whole application on a random port with the test profile (H2 database) and test accounts for staff, student and recruiter. One test walks through the main business flow by calling the real REST APIs:

- unauthenticated access to /api/students returns 401;
- an invalid student returns 400 with fieldErrors; a valid student is created (201) and updated (200);
- a company and an opportunity are created and updated by staff;
- the student applies (201) and a second application to the same opportunity returns 409;
- the recruiter sees exactly one application, changes its status to SELECTED, then schedules, updates and lists an interview;
- the staff dashboard shows 1 student, 1 application, 1 selected student and 1 upcoming interview;
- the student gets 403 for the student list, can read /api/students/me and sees the status SELECTED;
- the interview, application, opportunity, company and student are deleted in the correct order (204 each).

```
Set-Location .\backend
.\mvnw.cmd test
```

After this test passed, the same APIs were tested in detail with Postman (Industrial Assignment 12).

11. Coverage of the 28 Tasks

The table shows how each task of Industrial Assignment 1 is covered by the implemented backend. "Implemented" means the API and rules exist; "Partly" means a simpler version exists; "Future" means the feature is planned for a later version.

Task	Name	Status	How it is covered
T01	Register User	Partly	Staff create student profiles (POST /api/students); login accounts are configured at start-up. No public self-registration.
T02	User Login	Implemented	HTTP Basic with BCrypt; GET /api/auth/me returns user and roles.
T03	User Logout	Partly	Stateless API; the frontend clears the stored credentials on sign out.
T04	Manage User Access	Partly	Three roles with URL and method rules; accounts are managed through configuration.
T05	Create Student Profile	Implemented	POST /api/students (staff) with validation and duplicate checks.
T06	View Student Profile	Implemented	GET /api/students/{id} (staff) and GET /api/students/me (student).
T07	Edit Student Profile	Implemented	PUT /api/students/{id}; change history is Future.
T08	Upload and Manage Resume	Partly	resumeReference field stores the resume link; file upload is Future.
T09	Create Company Profile	Implemented	POST, PUT, DELETE /api/companies (staff), unique e-mail.
T10	Post Opportunity	Implemented	POST /api/opportunities by staff or the recruiter of the company.
T11	Edit Opportunity	Implemented	PUT /api/opportunities/{id} with ownership check; edit history is Future.
T12	Close or Delete Opportunity	Implemented	Status CLOSED through update; DELETE removes the record (blocked when applications exist).
T13	Apply for Opportunity	Implemented	POST /api/applications with open, deadline and duplicate checks.

Task	Name	Status	How it is covered
T14	View Application Details	Implemented	GET /api/applications scoped to student, recruiter or staff.
T15	Update Application Status	Implemented	PUT /api/applications/{id}/status with six statuses; status history is Future.
T16	Withdraw Application	Future	No WITHDRAWN status yet; staff can delete an application.
T17	Schedule Interview	Implemented	POST /api/interviews by staff or the recruiter of the company.
T18	View Interview Schedule	Partly	GET /api/interviews for staff and recruiters; student view is Future.
T19	Release Offer	Partly	Status SELECTED marks the result; a separate offer record is Future.
T20	Respond to Offer	Future	Accept or reject by the student is planned.
T21	View Dashboard	Implemented	GET /api/dashboard (staff).
T22	View Opportunity Summary	Implemented	activeInternships, activePlacements, opportunitiesByType in the dashboard.
T23	View Placement Summary	Implemented	selectedStudents, applicationsByStatus, placementOutcomes.
T24	Generate Reports	Partly	GET /api/reports returns a live summary; PDF or Excel export is Future.
T25	Search Opportunities	Partly	List and view opportunities; keyword search is Future.
T26	Filter Opportunities	Partly	Filter by type (?type=); students see only active, unexpired ones.
T27	Search Student Records	Partly	GET /api/students lists all students for staff; search parameters are Future.
T28	Filter Applications	Partly	Applications are filtered by role scope; filter parameters are Future.

Summary: 15 tasks are implemented, 11 are partly implemented with a simpler version and 2 are planned for a future version.

12. Build and Run

```
Set-Location .\backend
.\mvnw.cmd test       # runs SipmpApiIntegrationTest
.\mvnw.cmd package    # creates target\sipmp-backend-1.0.0.jar
.\mvnw.cmd spring-boot:run
```

Check after start: GET http://localhost:8080/api/auth/me with a Basic authorization header must return the user name and role.

13. Limitations and Future Enhancements

- Public registration, approval of recruiter accounts and a users table with session history.
- Resume file upload, student edit history, application status history and opportunity edit history.
- Withdraw application, offer release and offer response by the student.
- Eligibility checks on CGPA, branch and semester (needs these fields in the student profile).
- Keyword search and filters for opportunities, students and applications; paging for large lists.
- Report export to PDF and Excel and a rule that limits the allowed status changes of an application.
- Moving all secrets to environment variables and using a managed identity service in production.

14. Conclusion

The services, controllers and exception handling complete the Spring Boot backend of the portal. The backend offers role-protected REST APIs for students, companies, opportunities, applications, interviews, the dashboard and reports; it applies the rules for duplicate, closed and unauthorised requests; and it returns clear JSON errors. The integration test confirmed the main flow, and Industrial Assignment 12 tests every API in detail with Postman.